

Aufgabenblatt 2 – Kooperative Transportlogistik

Gruppennummer: Gruppe 11 · Bearbeiter: Ivan Ilin, Myron Sydorov, Philipp Spindler

1. Einleitung

Mehrere Fertigungsstandorte (Zentralen) müssen mit Items der Typen A, B, C aus verschiedenen Lagern versorgt werden. An jeder Zentrale agiert ein **Transportagent (TA)**, der eine möglichst kurze Rundreise zu den benötigten Lagern plant. Jedes Lager wird durch einen **Lageragenten (LA)** verwaltet. Das globale Ziel ist die Minimierung des Gesamtenergieverbrauchs (Summe aller Rundreise-Weglängen zuzüglich Konventionalstrafen von 100 Einheiten je unerfülltem Item).

Die Aushandlung erfolgt durch zwei Protokollvarianten. Im **CNP** versteigert jeder LA seine Items sequenziell; TA bieten mit ihrer marginalen Wegverlängerung und erhalten endgültige Zuschläge. Im **eCNP** laufen alle Auktionen parallel mit einer vorläufigen und einer definitiven Verhandlungsphase, was Umplanung ohne feste Reihenfolge ermöglicht.

2. Umsetzung

2.1 Kooperative Agenten

Transportagenten sind in `src/agents.py` als Klasse `TransportAgent` realisiert. Jeder TA kennt seinen eigenen Auftrag sowie die Koordinaten aller Lager (vollständige Information). Er verwaltet `committed` (definitiv kontraktierte Lager je Itemtyp); das **Marginalgebot** für ein weiteres Lager ist die Zunahme der Rundreisekosten bei dessen Hinzunahme (`marginal_bid`, Zeile 34–38). **Lageragenten** besitzen keine eigene Klasse; ihre Rolle wird durch die Protokollschleifen in `cnp.py` und `ecnp.py` übernommen. Die gesamte Kommunikation läuft über einen simulierten Nachrichtenbus (`Bus`, `messaging.py`) mit FIPA-orientierten Performativen.

2.2 Contract Net Protocol

Pro Durchlauf iteriert `cnp.py` über alle Lager und Itemtypen mit Bestand. LA sendet `cfp` an alle TA, die den Typ noch benötigen; TA antworten mit ihrem aktuellen Marginalgebot; LA erteilt den günstigsten Bietern den Zuschlag (`accept-proposal`) und weist die übrigen ab. Nach jedem Zuschlag ruft der TA `commit()` auf, sodass Folgegebote den bereits geplanten Pfad berücksichtigen (Synergie: zweites Item aus demselben Lager → Gebot = 0). Die Schleife wiederholt sich bis zur Konvergenz (kein Zuschlag mehr wechselt, max. 20 Pässe).

2.3 Erweitertes Contract Net Protocol

Das eCNP (`ecnp.py`) startet **alle Protokollinstanzen gleichzeitig** (4 Lager × 3 Itemtypen = 12 Instanzen). Jede Runde durchläuft fünf Phasen: (1) Broadcast-CFP aller offenen Instanzen, (2) parallele Marginalgebote aller TA, (3) `provisional-accept` an den besten Bieter bzw. `provisional-reject` (inkl. bestem Gebot) an die übrigen, (4) `definitive-bid` für die attraktivste vorläufige Zusage je Itemtyp plus `cancel` für konkurrierende Zusagen, (5) `definitive-accept` an das beste definitive Gebot. Der entscheidende Unterschied zum CNP: Endgültige Bindungen entstehen erst nach der zweiten Phase, was parallele Konfliktauflösung ohne neue Durchläufe erlaubt.

3. Analyse der Screencasts

Die Screencasts wurden ohne Tonspur aufgenommen; das Verhalten wird daher nachfolgend ausführlich beschrieben.

3.1 Experiment 1 – `cnp\_bal.mov`

Szenario: `balanced-9x9`; 4 Lager in den Ecken (je {A:1,B:1,C:1}), 4 TA (TA1 (1,6), TA2 (4,9), TA3 (9,4), TA4 (4,1)), Angebot = Nachfrage = 4 je Typ. Nicht-trivial, da TA1 und TA2 beide Lager LA3 (1,9) präferieren (Manhattan-Abstand = 3, Gebot = 6,0).

Im Screencast `cnp_bal.mov` ist ab [Zeitstempel, ca. 00:05] zu sehen, wie TA4 alle drei Items von LA1 in Folge gewinnt: B und C mit Gebot 0,0 (Synergie – er fährt ohnehin dorthin). Ab [Zeitstempel, ca. 00:30] wird der Engpass um LA3 sichtbar: TA1 und TA2 bieten identisch 6,0; LA3 erteilt TA1 den Zuschlag (ID-Tie-Breaking). TA2 erhält LA4 als verbleibendes Lager (Gebot 10,0). Durchlauf 2 zeigt `keine Aenderung -> Konvergenz`.

Ergebnis: TA1→LA3, TA2→LA4, TA3→LA2, TA4→LA1. Fahrenergie 28,0; Strafeenergie 0; **Gesamtenergie 28,0** (global optimal); 90 Nachrichten.

3.2 Experiment 2 – `cnp\_over.mov`

Szenario: `demand-overhang-9x9`; identische Lager, aber 5 TA (zusätzlich TA5 (5,5)). Nachfrage 5 > Angebot 4 je Typ – vollständige Versorgung strukturell unmöglich.

Im Screencast `cnp_over.mov` bietet TA5 in jeder Auktion konstant 16,0 (Abstand 8 zu jedem Ecklager). Ab [Zeitstempel, ca. 00:38] verliert TA5 auch die letzte Auktion (LA4) gegen TA2 (10,0 < 16,0). Die Zeile TA5: (leer) UNERFUELLT: A,B,C am Ende belegt das Ergebnis.

Ergebnis: TA1–TA4 identisch wie Exp. 1. Strafeenergie 300,0 (3 × 100); **Gesamtenergie 328,0** (strafminimal und optimal); 126 Nachrichten.

3.3 Experiment 3 – `ecnp\_bal.mov`

Szenario: `balanced-9x9` mit eCNP. Struktureller Unterschied sofort sichtbar.

Im Screencast `ecnp_bal.mov` erscheint ab [Zeitstempel, ca. 00:03] 12 parallele Protokollinstanzen gestartet. Ab [Zeitstempel, ca. 00:05] folgt ein dichter Block von 48 cfp-Nachrichten aller vier LA gleichzeitig – im CNP wären diese auf mehrere Auktionsblöcke verteilt. Ab [Zeitstempel, ca. 00:38] sind die vorläufigen Zusagen zu sehen: LA3 sendet `provisional-accept` an TA1 (6,0) und `provisional-reject` mit `bestes_gebot=6,0` an TA2 – TA2 kennt sofort den Marktpreis und kann in derselben Runde umplanen. Kein `cancel` erforderlich, da jeder TA genau eine vorläufige Zusage je Itemtyp erhielt. Das Protokoll endet nach einer einzigen Runde.

Ergebnis: Identische Zuteilung wie CNP. **Gesamtenergie 28,0**; 168 Nachrichten (87 % mehr als CNP).

3.4 Experiment 4 – `ecnp\_over.mov`

Szenario: `demand-overhang-9x9` mit eCNP.

Im Screencast `ecnp_over.mov` umfasst die cfp-Phase 60 Nachrichten (4 LA × 3 Typen × 5 TA). Ab [Zeitstempel, ca. 00:28] sind TA5s Gebote sichtbar: konstant 16,0 auf alle 12 Instanzen. Ab [Zeitstempel, ca. 00:48] erhält TA5 zwölf `provisional-reject`-Nachrichten und sendet kein einziges `definitive-bid`. Die Terminierungszeile lautet `keine offenen, benoetigten Instanzen mehr -> Ende`.

Ergebnis: Wie CNP-Overhang. **Gesamtenergie 328,0**; 204 Nachrichten.

Protokollvergleich

	CNP balanced	CNP overhang	eCNP balanced	eCNP overhang
Nachrichten	90	126	168	204
Gesamtenergie	28,0	328,0	28,0	328,0
Pässe/Runden	1	1	1	1

Beide Protokolle erreichen auf allen Instanzen das global optimale Ergebnis. eCNP löst Konflikte in einer Runde parallel, zahlt dafür aber mit höherem Nachrichtenaufwand.

4. Schwächen der Contracting-Lösung

Schwäche 1: Reihenfolgeabhängigkeit im CNP

Die äußere Schleife in `cnp.py` (Zeile 34) iteriert Lager in fester Reihenfolge (LA1 → LA2 → LA3 → LA4). Frühe Zuschläge binden TA endgültig, bevor alle Auktionen bekannt sind. In komplexeren Szenarien mit ungleichmäßig verteiltem Angebot kann ein früher, lokal optimaler Zuschlag zu einer global suboptimalen Lösung führen, da kein Backtracking möglich ist.

Verbesserungsvorschlag: Das eCNP-Prinzip der vorläufigen Zusagen überträgt sich direkt: Kontrakte werden erst nach Evaluierung aller parallelen Auktionen finalisiert. Alternativ kann ein *Leveled-Commitment*-Mechanismus Verträge gegen Strafgebühr auflösen, wenn eine bessere Gesamtlösung erkennbar wird.

Schwäche 2: Fehlende Reaktion auf `cancel` im eCNP

`cancel` wird in `ecnp.py` (Zeile 122–123) korrekt gesendet, aber der empfangende LA verarbeitet die Nachricht nicht: Er streicht den Sender nicht aus seiner Warteliste und sendet kein neues `provisional-accept` an den nächstbesten Bieter. In den gezeigten Experimenten blieb die Lücke folgenlos, weil kein TA mehrere Zusagen für denselben Itemtyp erhielt. Bei mehr konkurrierenden Lagern würde ein LA unnötig auf ein `definitive-bid` warten, das nie eintrifft.

Verbesserungsvorschlag: `Instance` sollte eine sortierte Warteliste der Bieter führen. Nach Empfang von `cancel` sendet LA sofort `provisional-accept` an den nächsten Kandidaten.

5. Brokering-Variante

In einer Broker-Architektur meldet jeder TA seinen Bedarf bei einem zentralen **Broker-Agenten** an. Der Broker aggregiert die Bestände aller LA, berechnet eine globale Zuteilung (z. B. per Integer-Programmierung) und übermittelt das Ergebnis verbindlich an TA und LA. Direkte TA↔LA-Kommunikation entfällt.

Vorteile: (1) Garantiert globale Optimalität, die CNP/eCNP nur annähern. (2) Drastisch weniger Nachrichten: statt $n \cdot m$ Paar-Interaktionen nur $n + m$ Nachrichten zum Broker.

Nachteile: (1) Single Point of Failure – Ausfall des Brokers legt das gesamte System lahm. (2) Verlust von Autonomie und Skalierbarkeit: Der Broker muss den Zustand aller Agenten kennen und wird bei wachsender Instanzgröße zum Flaschenhals.

6. Fazit

Beide Protokolle lösen das Transportlogistik-Problem korrekt und erreichen auf den getesteten Instanzen das globale Optimum. **CNP** ist kommunikationseffizienter (90–126 Nachrichten) und direkt verständlich, aber durch seine sequenzielle Vergabe potenziell reihenfolgeabhängig. **eCNP** ist robuster durch parallele Ausführung und zweistufige Bindung, hat aber deutlich höheren Nachrichtenaufwand (168–204 Nachrichten); auf den symmetrischen Testinstanzen ergibt sich kein Qualitätsvorteil. Eine **Broker-Variante** böte garantierte Optimalität, opfert jedoch Fehlertoleranz und Dezentralität – die wesentlichen Stärken eines Multiagentenansatzes.